

RAG-Based Email Customer Support Agent

Architecture: Gmail Trigger → Data Prep → AI Agent (RAG) → Vector Retrieval → Reply Generation → Send / Approve

Apu Ghosh

1. System Workflow

The system automates the full customer-support email cycle — from the moment an email arrives to the moment a reply is sent — using an n8n workflow built around a Retrieval-Augmented Generation (RAG) architecture.

Step-by-Step Flow

1. **Email Received:** A Gmail Trigger node continuously watches the support inbox and fires whenever a new customer email arrives.
2. **Data Extraction:** An Edit Fields node pulls out the key data — sender, subject, message body, thread ID — from the raw email payload.
3. **Pre-processing:** A Code (JavaScript) node cleans and formats this data (strips signatures/HTML, normalizes text) so it is ready for the AI Agent.
4. **AI Agent Invocation:** The formatted query is passed to an AI Agent node, which is connected to an LLM (OpenAI Chat Model), a memory module, and a retrieval tool.
5. **Knowledge Retrieval (RAG):** The agent's tool embeds the customer's question and performs a similarity search against a vector database (Supabase Vector Store) containing the company's knowledge base.
6. **Context-Aware Generation:** The retrieved knowledge is injected into the LLM's prompt, and the model generates an accurate, grounded reply instead of relying purely on its own training data.
7. **(Optional) Human Approval:** The draft reply can be routed to a human agent for review before sending (see Bonus Task).
8. **Reply Sent:** A Gmail node sends the generated reply back to the customer in the same email thread.
9. **Mark as Read:** The original email is marked as read/processed so it is not picked up again.

2. Tool & Platform Selection

The following stack was chosen for reliability, low setup overhead, and native RAG support:

Layer	Tool / Platform	Purpose
Workflow Orchestration	n8n	Connects all nodes: trigger, retrieval, AI agent, reply, approval
Email Trigger & Delivery	Gmail API / Gmail Trigger	Receives new customer emails and sends the final reply
LLM (Reasoning & Generation)	OpenAI API (GPT-4 / GPT-4o class model)	Understands the query and drafts the reply
Embeddings	OpenAI Embeddings (text-embedding-3, 1536-dim)	Converts text (KB docs + query) into vectors
Vector Database	Supabase Vector Store (pgvector) — alt: Pinecone / ChromaDB	Stores embeddings and performs similarity search
Memory	Simple Memory / Buffer Memory (n8n)	Keeps short conversational context per thread
Orchestration Logic	LangChain (concepts used inside n8n's AI Agent node)	Chains retrieval → prompt → generation
Human-in-the-Loop	n8n Approval node / Slack / Email notification	Lets a support agent review before sending

Note: Supabase Vector Store can be swapped for Pinecone or ChromaDB with no change to the overall architecture — only the vector-store node changes.

3. Email Processing

Received

- The Gmail Trigger node polls (or uses push notifications via Gmail API) for new messages in the support inbox.
- Each new email is captured as a structured JSON item containing id, threadId, sender, subject, body, and labels.

Processed

- The Edit Fields node maps only the relevant fields (sender email, subject, body) into a clean object.
- The Code node strips HTML tags, quoted reply chains, and email signatures, and normalizes whitespace/encoding.
- Basic validation checks (e.g., non-empty body, spam/label filtering) can be added here to skip promotional or irrelevant emails.

Prepared for AI Analysis

- The cleaned text is combined into a single "customer query" string.
- This becomes the input passed to the AI Agent, along with metadata (thread ID, sender) preserved for the reply step.

4. Information Retrieval (RAG)

Before answering, the agent retrieves the most relevant company information instead of relying only on the LLM's general knowledge. This keeps answers accurate, current, and specific to the business.

Knowledge Base

- Company documents — FAQs, refund/return policy, product manuals, past resolved tickets — are collected as the source knowledge base.

Embeddings

- Each document chunk is converted into a 1536-dimension vector using OpenAI's embedding model (Embeddings OpenAI node).
- The same embedding model is used at query time to embed the customer's question, so both live in the same vector space.

Vector Database

- All document embeddings are stored in a Supabase Vector Store (pgvector), indexed for fast nearest-neighbor lookup.

Similarity Search

- When a query arrives, its embedding is compared against stored embeddings using cosine similarity.
- The top-matching chunks (e.g., top 2) are retrieved and returned to the AI Agent as context — this is the "Answer questions with..." tool seen in the workflow.

Augmented Generation

- The retrieved chunks are inserted into the LLM prompt alongside the original question, so the model generates its answer grounded in real company data rather than guessing.

5. Response Generation & Delivery

Generating the Reply

- The AI Agent (OpenAI Chat Model) combines the customer's question, retrieved knowledge-base context, and short-term conversation memory to draft a natural, on-brand reply.
- System-prompt instructions constrain tone (polite, professional) and require the model to say "I don't have that information" rather than hallucinate when no relevant context is found.

Sending the Response

- The drafted reply is passed to a Gmail "Reply to a message" node, which sends it within the original thread so the customer sees a normal email conversation.
- A final "Mark as read" node updates the email's label/status so it is not reprocessed by the trigger.

6. Real-World Use Cases

Use Case	How RAG Email Agent Helps
E-commerce Customer Support	Answers order status, return/refund policy, and product questions by retrieving from order and policy documents
SaaS Product Support	Resolves how-to and troubleshooting queries using product documentation and past support tickets as the knowledge base
Banking / Fintech Support	Answers account, fee, and service queries from official policy documents, with mandatory human approval before sending
Educational Platforms	Answers admission, course, and fee-structure queries from institutional handbooks and FAQs
Healthcare Administrative Queries	Handles appointment, billing, and general policy questions (non-clinical) from hospital/clinic documentation

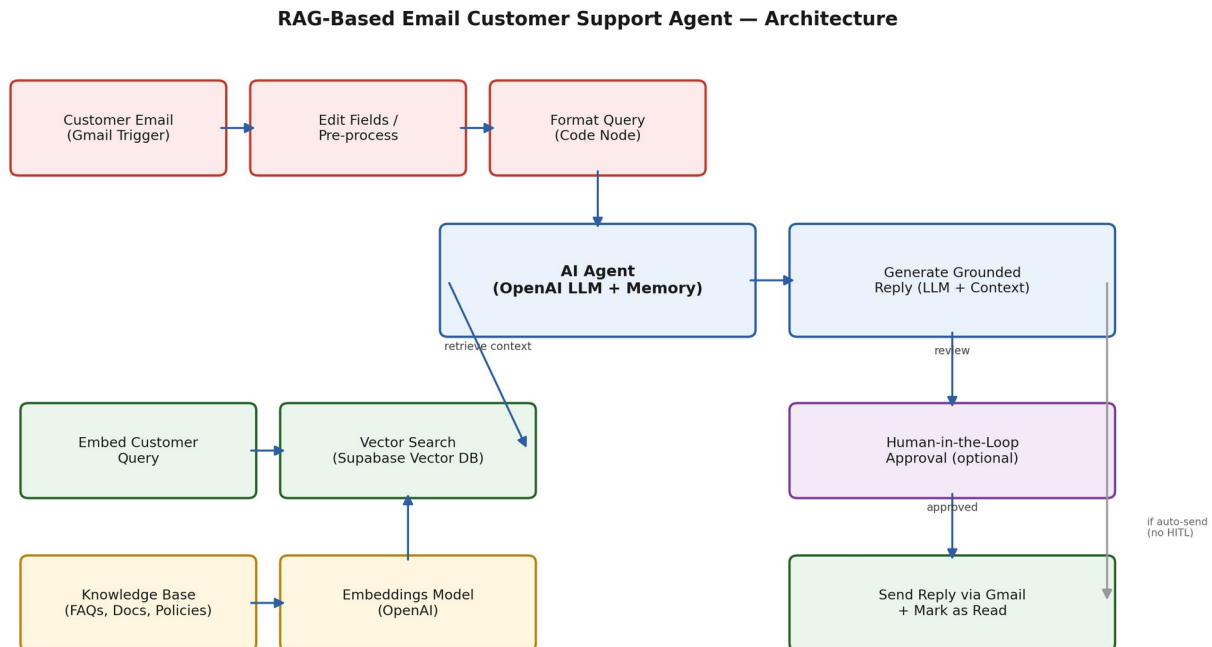
7. Challenges & Limitations

Challenge	Description & Mitigation
Hallucination	The LLM may generate a confident but incorrect answer when the knowledge base lacks relevant content. Mitigated by strict prompt instructions to answer only from retrieved context and to escalate if no match is found
Outdated Knowledge Base	If company policies change but embeddings are not refreshed, the agent gives stale answers. Mitigated by a scheduled re-embedding pipeline whenever source documents are updated
Privacy & Security	Customer emails may contain personal or financial data sent to a third-party LLM API. Mitigated by data redaction, encryption in transit, and vendor DPAs / compliance review
Complex / Ambiguous	Multi-intent or emotionally sensitive emails (complaints, legal issues) may need

Queries	a human touch. Mitigated by confidence scoring and automatic escalation to a human agent
API & Infrastructure Cost	Embedding + LLM calls per email add up at scale. Mitigated by caching common Q&A pairs and using smaller/cheaper models for simple queries

8. Workflow Architecture Diagram

The diagram below summarizes the end-to-end flow: email intake, RAG retrieval from the knowledge base, grounded response generation, an optional human-approval checkpoint, and final delivery.



★ Bonus Task: Human-in-the-Loop (HITL) Approval

To reduce risk in sensitive domains (finance, healthcare, legal, or any negative/emotional customer email), a human-review checkpoint can be inserted before the reply is sent.

Human Review Process

- **Draft, don't send:** Instead of sending the reply directly, the AI Agent's output is routed to a review step (e.g., n8n's Wait-for-Approval node, or a Slack/Email notification to the support team).
- **Review interface:** The support agent sees the customer's original email, the AI-drafted reply, and the retrieved knowledge-base context used to generate it — enabling an informed decision.

Approval / Rejection Flow

- **Approve:** The agent clicks "Approve" (or replies "yes" in Slack); the workflow resumes and proceeds to the Gmail send step unchanged.

- **Edit & Approve:** The agent can edit the draft text before approving; the edited version is what gets sent.
- **Reject:** The agent rejects the draft; the workflow either loops back to regenerate a new draft with additional instructions, or escalates the email to be answered manually outside the automation.

Sending Only After Approval

- The Gmail "Reply to a message" node is placed after the approval branch, so it only executes on the "approved" path — the workflow's IF/Switch logic gates delivery on the human decision.
- This turns the fully automated pipeline into a safer "AI drafts, human decides" pipeline, which is recommended for production use until the system's accuracy is proven over time.

— *End of Submission* —